STAT 443 Project

```
library(forecast)
```

Functions

Some of the functions below were taken from labs or lectures.

```
# Function for variogram as defined in
# Bisgaard and Kulahci (2011). Time Series Analysis and Forecasting by Example, Wiley
variogram = function(y, lagmax=10, iprint=F)
{ G = rep(1,lagmax)
n = length(y)
if(lagmax>n) { lagmax = n-2 }
y1 = y[-1]; y2 = y[-n]
d1 = y1-y2; denom = var(d1)
for(k in 2:lagmax)
{ y1 = y[(k+1):n]; y2 = y[1:(n-k)]
dk = y1-y2
numer = var(dk)
G[k] = numer/denom
}
# H is the variogram assuming a stationary time series
H = rep(1,lagmax)
ac = c(acf(y,plot=F,lag.max=lagmax)$acf)
H = (1-ac[-1])/(1-ac[2])
if(iprint)
{ print(cbind(G,H))
mx = max(cbind(G,H))
matplot(1:lagmax,cbind(G,H),ylim=c(0,mx),ylab="variogram", xlab="lag")
abline(h=1)
}
```

```
list(G=G, H=H)
}
```

```
persist_fc = function(train, holdout, iprint = FALSE) {
  ntrain = length(train)
  nh = length(holdout)
  sse = 0
  fc_vec = numeric(nh)
  fc = train[ntrain]
  yt = holdout[1]
  fc_vec[1] = fc
  fcerror = yt - fc
  sse = sse + (fcerror^2)
  for (i in 2:nh) {
    yt = holdout[i]
    fc = holdout[i-1]
    fc_vec[i] = fc
    fcerror = yt - fc
    sse = sse + (fcerror^2)
  }
  return(list(rmse = sqrt(sse/nh), fc = fc_vec))
}

# Linear in the previous observation
# b0,b1 from least squares or another estimation method
linreg_fc = function(train_last, holdout, b0,b1)
{ nh = length(holdout)
  previous = c(train_last, holdout[-nh])
  fc_vec = b0+b1*previous
  rmse = sqrt(mean((holdout-fc_vec)^2))
  return(list(rmse=rmse, fc=fc_vec))
}

# Average Forecast
avg_fc = function(train, holdout) {
  nh = length(holdout)
  avg = mean(train)
  sse = 0
  fc_vec = numeric(nh)
  for (i in 1:nh) {
    fc = avg
    fc_vec[i] = fc
```

```r
    yt = holdout[i]
    fcerror = yt - fc
    sse = sse + (fcerror^2)
  }
  return(list(rmse = sqrt(sse/nh), fc = fc_vec))
}

# Simple Exponential Smoothing
esm_fc = function(train, holdout, alpha, level, iprint = FALSE) {
  nh = length(holdout)
  sse = 0
  fc_vec = numeric(nh)
  L = level
  for (i in 1:nh) {
    fc = L
    fc_vec[i] = fc
    yt = holdout[i]
    fcerror = yt - fc
    sse = sse + (fcerror^2)
    L = alpha * yt + (1 - alpha) * L
  }
  return(list(rmse = sqrt(sse / nh), fc = fc_vec))
}
```

```r
plot_forecast = function(holdout, fc, model_name = "Model") {

  x = seq_along(holdout)

  plot(x, holdout, type = "l", col = "black", lwd = 2,
       main = paste("Forecast vs Actual -", model_name),
       ylab = "Value", xlab = "Time")

  lines(x, fc, col = "red", lwd = 2)

  legend("topleft",
         legend = c("Actual", "Forecast"),
         col = c("black", "red"),
         lty = 1, lwd = 2)
}
```

```r
arima_rolling_fc = function(fitted_model, holdout, iprint = FALSE) {
  nh = length(holdout)
  sse = 0
  fc_vec = numeric(nh)

  # Extract the training data from the fitted model
  train_data = fitted_model$x
  current_data = train_data

  for (i in 1:nh) {
    # Forecast one step ahead using current model
    fc = forecast(Arima(current_data, model = fitted_model), h = 1)$mean[1]
    fc_vec[i] = fc

    yt = holdout[i]
    fcerror = yt - fc
    sse = sse + (fcerror^2)

    # Expand training data with actual observed value
    current_data = c(current_data, yt)
  }

  return(list(rmse = sqrt(sse / nh), fc = fc_vec))
}
```

```r
rolling_armax_fixed = function(y_train, x_train, y_holdout, x_holdout,
                               order, include_mean = TRUE) {
  nh = length(y_holdout)
  fc_vec = numeric(nh)

  y_hist = y_train
  x_hist = as.matrix(x_train)
  x_holdout = as.matrix(x_holdout)

  for (i in 1:nh) {
    fit = forecast::Arima(
      y_hist,
      order = order,
      xreg = x_hist,
      include.mean = include_mean,
      method = "ML"
    )
```

```
    newx = matrix(x_holdout[i, ], nrow = 1)

    fc_vec[i] = forecast::forecast(
      fit,
      xreg = newx,
      h = 1
    )$mean[1]

    y_hist = c(y_hist, y_holdout[i])
    x_hist = rbind(x_hist, newx)
  }

  rmse = sqrt(mean((y_holdout - fc_vec)^2))
  list(rmse = rmse, fc = fc_vec)
}
```

Load Data

```
# Load Spot Crude Oil Price
crude_oil = read.csv("data/bloomberg_oil_spot_monthly.tsv", header = TRUE, sep = "\t")
crude_oil$Date = as.Date(crude_oil$Date)
crude_oil = crude_oil[order(crude_oil$Date), ]

crude_oil_ts = ts(
  crude_oil$Last.Price,
  start = c(as.numeric(format(min(crude_oil$Date), "%Y")),
            as.numeric(format(min(crude_oil$Date), "%m"))),
  frequency = 12
)

head(crude_oil_ts)
```

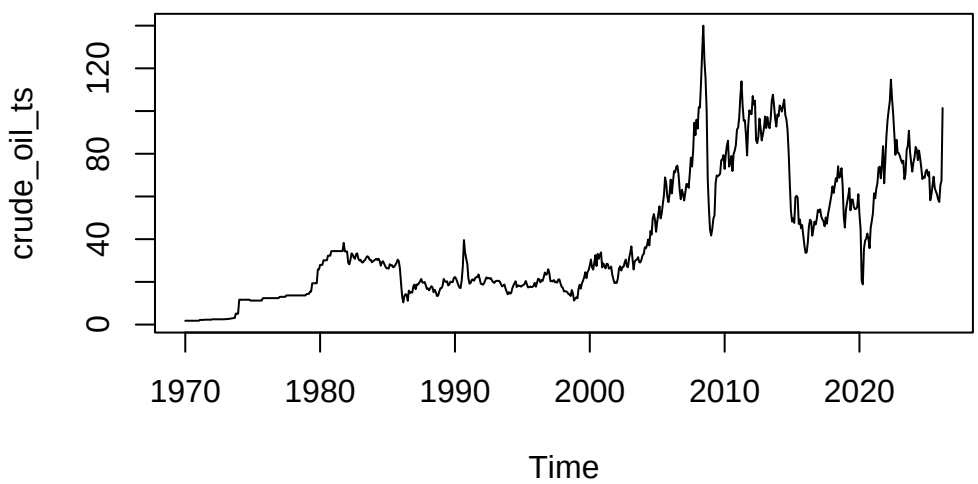
```
     Jan Feb Mar Apr May Jun
1970 1.8 1.8 1.8 1.8 1.8 1.8
```

Split Training - Holdout

```r
crude_oil_train_ts = window(crude_oil_ts, start = c(1975,1), end = c(2010,12))
crude_oil_holdout_ts = window(crude_oil_ts, start = c(2011,1), end = c(2026,1))

d_crude_oil_train_ts = diff(crude_oil_train_ts)
d_crude_oil_holdout_ts = diff(crude_oil_holdout_ts)

dl_crude_oil_ts = diff(log(crude_oil_ts))
dl_crude_oil_train_ts = diff(log(crude_oil_train_ts))
dl_crude_oil_holdout_ts = diff(log(crude_oil_holdout_ts))
```

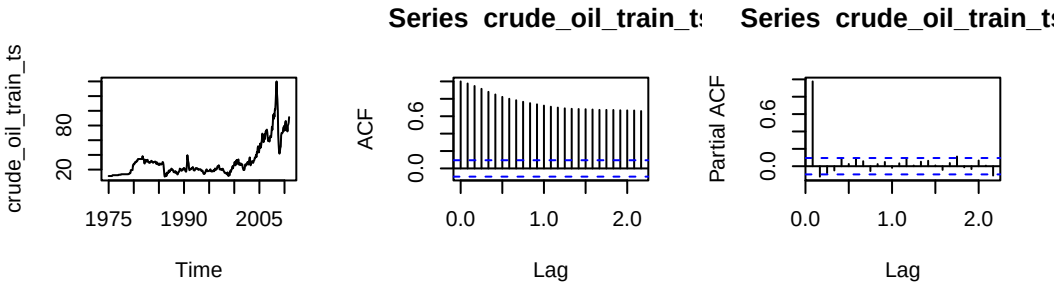
Diagnostic Plots

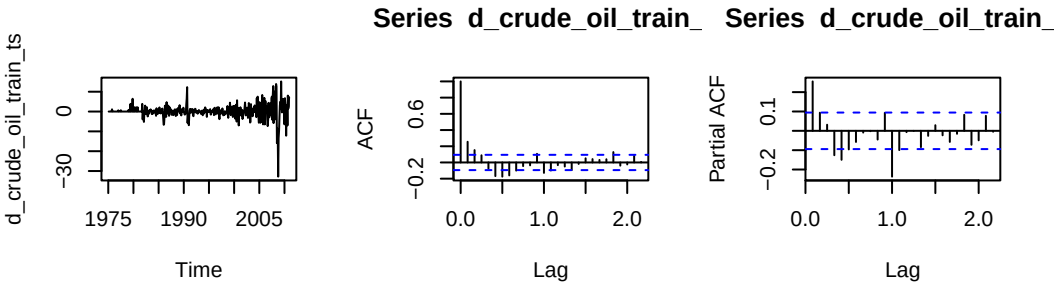
```r
plot(crude_oil_ts)
```



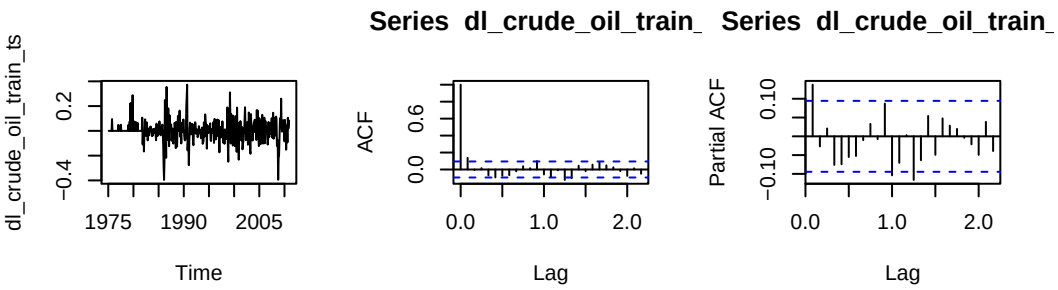

```r
par(mfrow = c(2,3))

plot.ts(crude_oil_train_ts)
acf(crude_oil_train_ts)
pacf(crude_oil_train_ts)
```

```r
plot.ts(d_crude_oil_train_ts)
acf(d_crude_oil_train_ts)
pacf(d_crude_oil_train_ts)
```





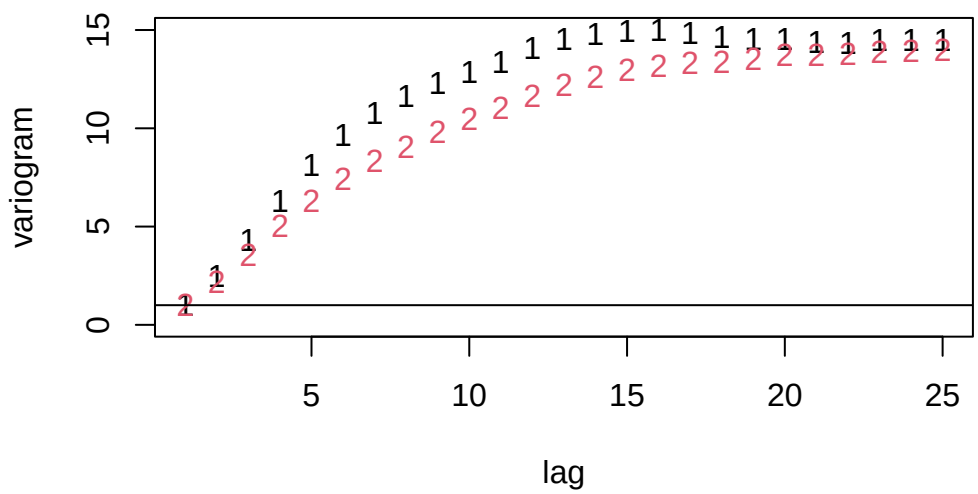

```r
plot.ts(dl_crude_oil_train_ts)
acf(dl_crude_oil_train_ts)
pacf(dl_crude_oil_train_ts)
```




```r
variogram(crude_oil_train_ts, lagmax=25, iprint=T)
```

```
              G         H
 [1,]  1.000000  1.000000
 [2,]  2.507923  2.213314
 [3,]  4.322784  3.575911
 [4,]  6.320108  5.027343
 [5,]  8.134092  6.293844
 [6,]  9.654462  7.451824
 [7,] 10.812447  8.365963
 [8,] 11.642819  9.074476
 [9,] 12.316291  9.817642
[10,] 12.897735 10.476988
[11,] 13.397260 11.046448
[12,] 14.086429 11.660209
[13,] 14.545171 12.204713
[14,] 14.793300 12.609311
[15,] 14.972980 12.973769
[16,] 15.014611 13.202572
[17,] 14.856925 13.314807
[18,] 14.657656 13.401482
[19,] 14.568813 13.556345
```

```
[20,] 14.541040 13.718163
[21,] 14.416240 13.731384
[22,] 14.355121 13.774623
[23,] 14.491999 13.909540
[24,] 14.497082 13.961572
[25,] 14.500442 14.019781
```

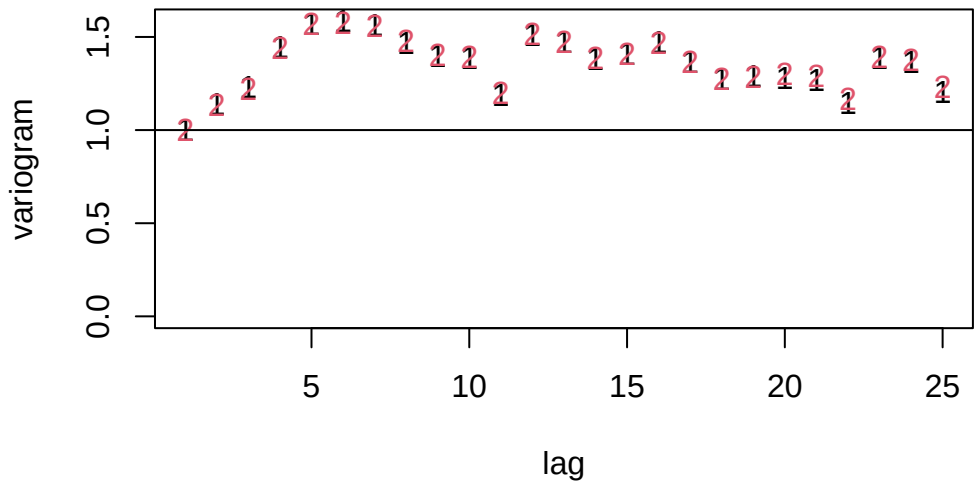



```
$G
 [1]  1.000000  2.507923  4.322784  6.320108  8.134092  9.654462 10.812447
 [8] 11.642819 12.316291 12.897735 13.397260 14.086429 14.545171 14.793300
[15] 14.972980 15.014611 14.856925 14.657656 14.568813 14.541040 14.416240
[22] 14.355121 14.491999 14.497082 14.500442

$H
 [1]  1.000000  2.213314  3.575911  5.027343  6.293844  7.451824  8.365963
 [8]  9.074476  9.817642 10.476988 11.046448 11.660209 12.204713 12.609311
[15] 12.973769 13.202572 13.314807 13.401482 13.556345 13.718163 13.731384
[22] 13.774623 13.909540 13.961572 14.019781
```

```r
variogram(d_crude_oil_train_ts, lagmax=25, iprint=T)
```

```
              G         H
```

```
 [1,] 1.000000 1.000000
 [2,] 1.140406 1.137681
 [3,] 1.230013 1.223906
 [4,] 1.448366 1.443887
 [5,] 1.572692 1.568983
 [6,] 1.584348 1.577965
 [7,] 1.567000 1.557561
 [8,] 1.471277 1.478062
 [9,] 1.397354 1.403207
[10,] 1.386399 1.391114
[11,] 1.187448 1.199834
[12,] 1.511651 1.516234
[13,] 1.473859 1.476647
[14,] 1.385325 1.387892
[15,] 1.407840 1.410880
[16,] 1.472604 1.469787
[17,] 1.369459 1.367682
[18,] 1.274489 1.273881
[19,] 1.290106 1.287193
[20,] 1.279969 1.301071
[21,] 1.271118 1.289944
[22,] 1.145389 1.170839
[23,] 1.385722 1.395920
[24,] 1.366577 1.375749
[25,] 1.207327 1.234587
```

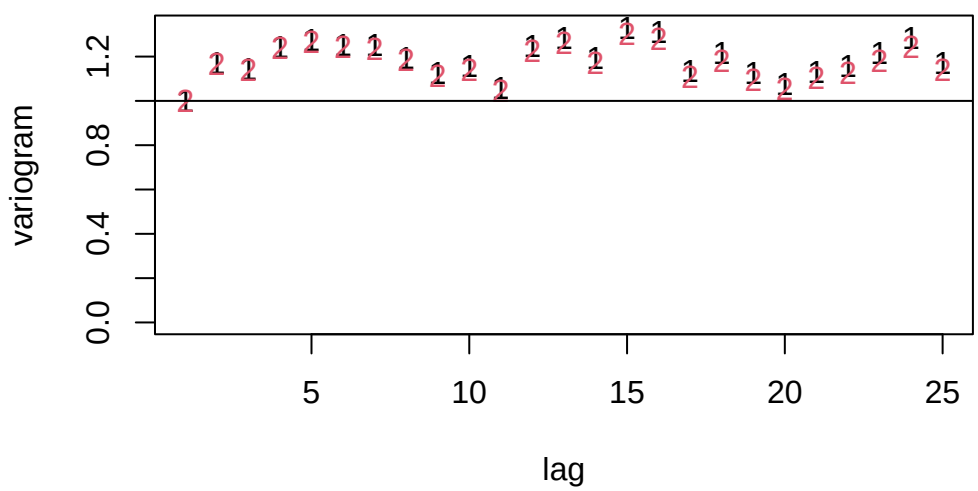



```
$G
 [1] 1.000000 1.140406 1.230013 1.448366 1.572692 1.584348 1.567000 1.471277
 [9] 1.397354 1.386399 1.187448 1.511651 1.473859 1.385325 1.407840 1.472604
[17] 1.369459 1.274489 1.290106 1.279969 1.271118 1.145389 1.385722 1.366577
[25] 1.207327

$H
 [1] 1.000000 1.137681 1.223906 1.443887 1.568983 1.577965 1.557561 1.478062
 [9] 1.403207 1.391114 1.199834 1.516234 1.476647 1.387892 1.410880 1.469787
[17] 1.367682 1.273881 1.287193 1.301071 1.289944 1.170839 1.395920 1.375749
[25] 1.234587
```

```r
variogram(dl_crude_oil_train_ts, lagmax=25, iprint=T)
```

```
             G        H
 [1,] 1.000000 1.000000
 [2,] 1.170684 1.167915
 [3,] 1.146258 1.140938
 [4,] 1.246487 1.239469
 [5,] 1.276368 1.267777
 [6,] 1.255360 1.244322
 [7,] 1.251255 1.237415
```

```
 [8,] 1.196123 1.184554
 [9,] 1.126131 1.114659
[10,] 1.156505 1.142099
[11,] 1.057318 1.043851
[12,] 1.246984 1.227380
[13,] 1.285195 1.261666
[14,] 1.194501 1.170825
[15,] 1.331962 1.302037
[16,] 1.310576 1.278328
[17,] 1.136224 1.107609
[18,] 1.217635 1.183091
[19,] 1.127386 1.094287
[20,] 1.078727 1.056457
[21,] 1.129810 1.102712
[22,] 1.158570 1.129139
[23,] 1.215641 1.181328
[24,] 1.286229 1.246404
[25,] 1.171995 1.142738
```




```
$G
 [1] 1.000000 1.170684 1.146258 1.246487 1.276368 1.255360 1.251255 1.196123
 [9] 1.126131 1.156505 1.057318 1.246984 1.285195 1.194501 1.331962 1.310576
```

```
[17] 1.136224 1.217635 1.127386 1.078727 1.129810 1.158570 1.215641 1.286229
[25] 1.171995

$H
 [1] 1.000000 1.167915 1.140938 1.239469 1.267777 1.244322 1.237415 1.184554
 [9] 1.114659 1.142099 1.043851 1.227380 1.261666 1.170825 1.302037 1.278328
[17] 1.107609 1.183091 1.094287 1.056457 1.102712 1.129139 1.181328 1.246404
[25] 1.142738
```

We first try ARMA

ARMA on difference

```r
dwti_arma = auto.arima(d_crude_oil_train_ts, seasonal=F, stationary=T)
print(dwti_arma)
```

```
Series: d_crude_oil_train_ts
ARIMA(2,0,2) with zero mean

Coefficients:
         ar1      ar2      ma1     ma2
      1.4117  -0.7988  -1.2275  0.7268
s.e.  0.0819   0.0655   0.0823  0.0733

sigma^2 = 11.95:  log likelihood = -1144.38
AIC=2298.75   AICc=2298.9   BIC=2319.09
```

Rolling 1-step forward

```r
dwti_arma_rolling = arima_rolling_fc(dwti_arma, d_crude_oil_holdout_ts)
dwti_arma_rolling$rmse
```

```
[1] 6.550004
```

ARMA on log return

```r
dlwti_arma = auto.arima(dl_crude_oil_train_ts, seasonal=F, stationary=T)
print(dlwti_arma)
```

```
Series: dl_crude_oil_train_ts
ARIMA(0,0,1) with zero mean

Coefficients:
         ma1
      0.1459
s.e.  0.0486

sigma^2 = 0.007729:  log likelihood = 436.85
AIC=-869.7   AICc=-869.68   BIC=-861.57
```

Rolling 1-step forward

```r
dlwti_arma_rolling = arima_rolling_fc(dlwti_arma, dl_crude_oil_holdout_ts)
dlwti_arma_rolling$rmse
```

```
[1] 0.1134567
```

ARMA based on ACF/PACF

```r
dlwti_arma_manual = Arima(dl_crude_oil_train_ts, order = c(0, 0, 1))
print(dlwti_arma_manual)
```

```
Series: dl_crude_oil_train_ts
ARIMA(0,0,1) with non-zero mean

Coefficients:
         ma1    mean
      0.1439  0.0049
s.e.  0.0487  0.0048

sigma^2 = 0.007729:  log likelihood = 437.36
AIC=-868.72   AICc=-868.67   BIC=-856.52
```

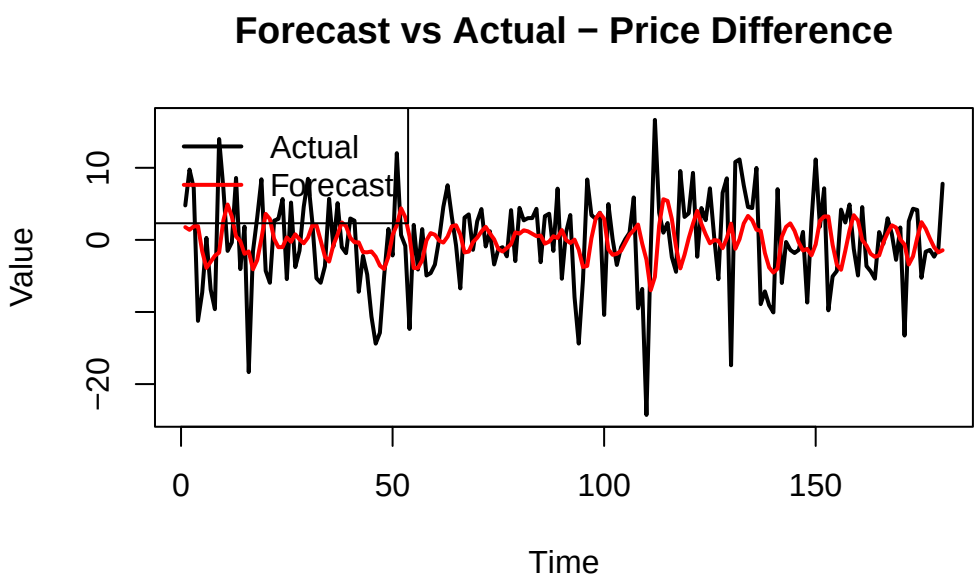
Rolling 1-step forward

```
dlwti_arma_manual_rolling = arima_rolling_fc(dlwti_arma_manual, dl_crude_oil_holdout_ts)
dlwti_arma_manual_rolling$rmse
```

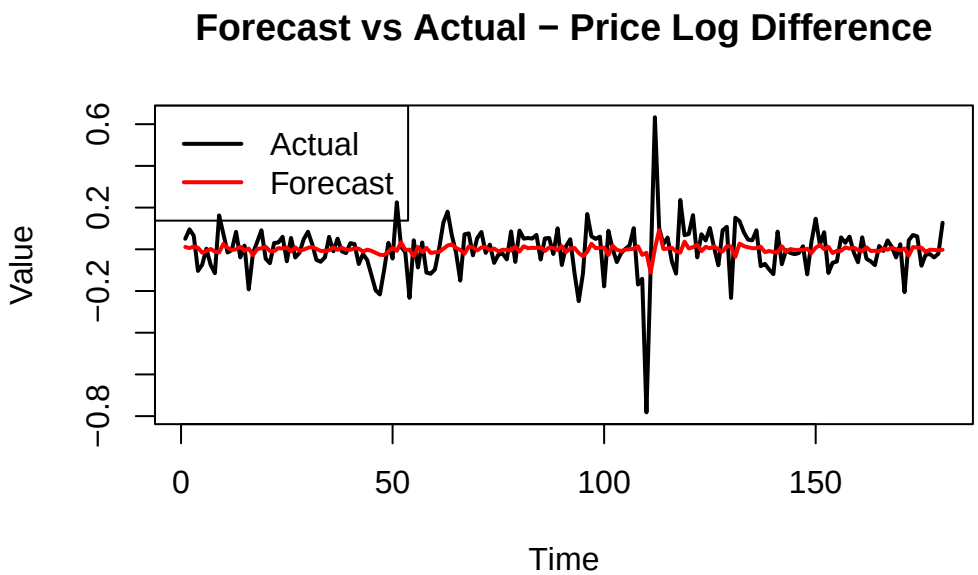
```
[1] 0.1136074
```

Plots

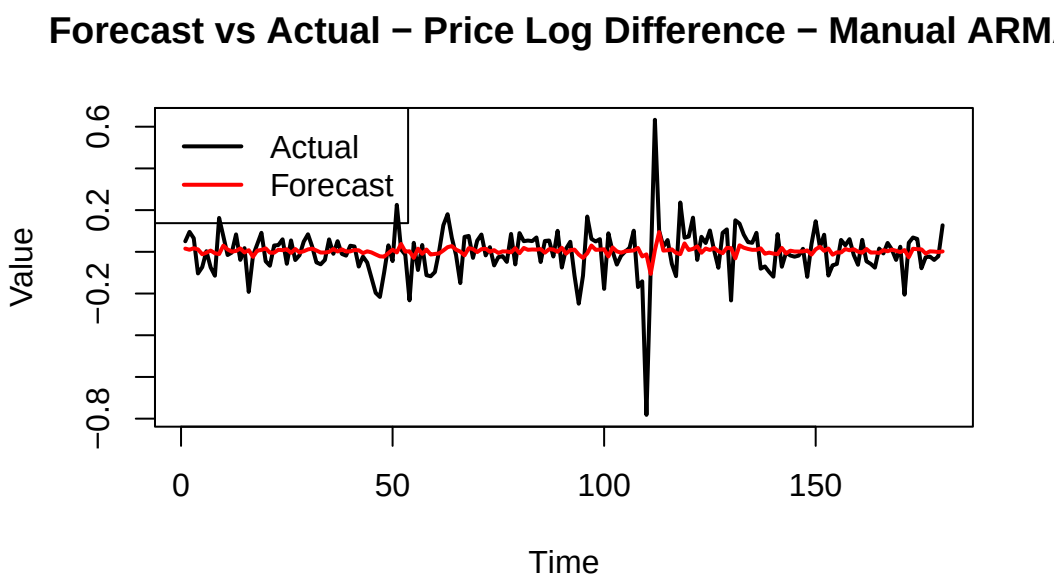
```
plot_forecast(d_crude_oil_holdout_ts, dwti_arma_rolling$fc, "Price Difference")
```




```
plot_forecast(dl_crude_oil_holdout_ts, dlwti_arma_rolling$fc, "Price Log Difference")
```




```
plot_forecast(dl_crude_oil_holdout_ts, dlwti_arma_manual_rolling$fc, "Price Log Difference -
```

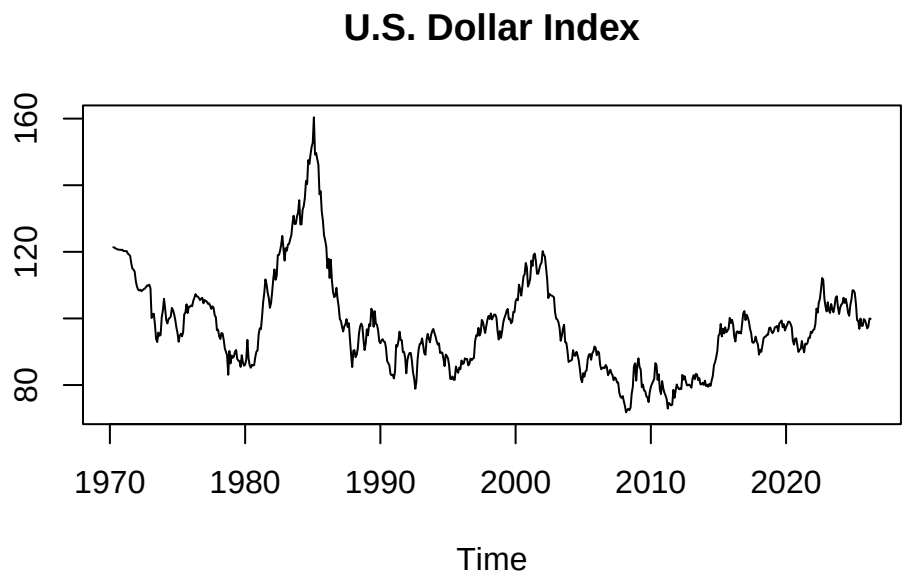


Extrogenous variables

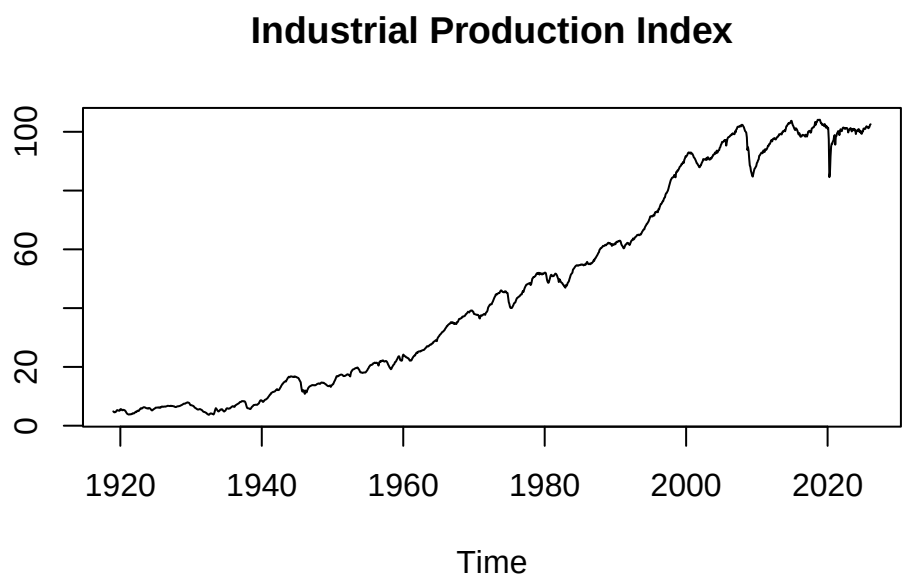
Load data

```
# USD index
usd_idx = read.csv("data/dxy_monthly.tsv", header = TRUE, sep = "\t")
usd_idx$Date = as.Date(usd_idx$Date)
usd_idx = usd_idx[order(usd_idx$Date), ]

usd_idx_ts = ts(
  usd_idx$Last.Price,
  start = c(1970,04),
  end = c(2026,04),
  frequency = 12
)
usd_idx_train_ts = window(usd_idx_ts, start = c(1975,1), end = c(2010,12))
usd_idx_holdout_ts = window(usd_idx_ts, start = c(2011,1), end = c(2026,1))

# Industrial production
ind_prod = read.csv("data/INDPRO.csv")
ind_prod$observation_date = as.Date(ind_prod$observation_date)
ind_prod_ts = ts(ind_prod$INDPRO, start = c(1919,1), end = c(2026,2), frequency = 12)
ind_prod_train_ts = window(ind_prod_ts, start = c(1975,1), end = c(2010,12))
ind_prod_holdout_ts = window(ind_prod_ts, start = c(2011,1), end = c(2026,1))

# OPEC oil production
opec_prod = read.csv("data/opcrtotl_monthly.tsv", header = TRUE, sep = "\t")
opec_prod$Date = as.Date(opec_prod$Date)
opec_prod = opec_prod[order(opec_prod$Date), ]

opec_prod_ts = ts(
  opec_prod$Last.Price,
  start = c(1970,12),
  end = c(2026,03),
  frequency = 12
)
opec_prod_train_ts = window(opec_prod_ts, start = c(1975,1), end = c(2010,12))
opec_prod_holdout_ts = window(opec_prod_ts, start = c(2011,1), end = c(2026,1))
```

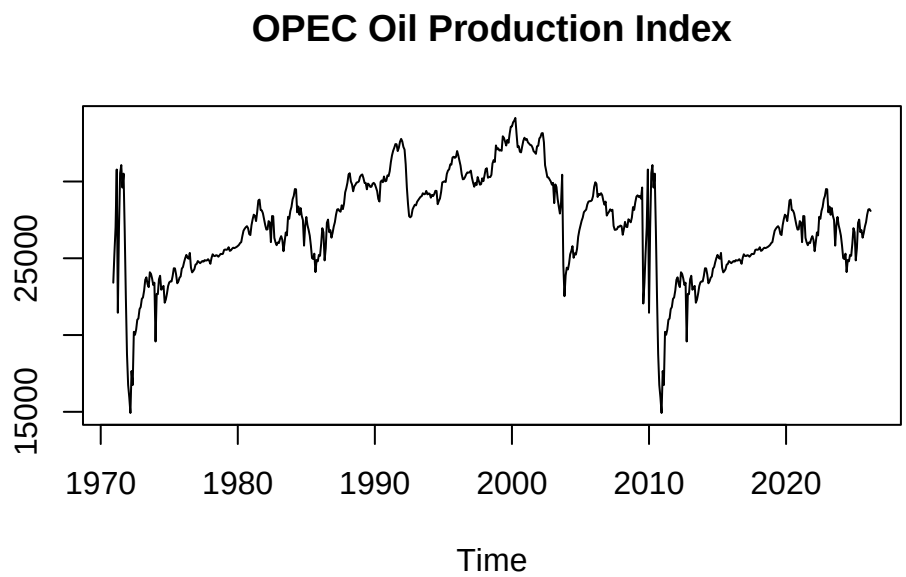
```
plot(usd_idx_ts, main="U.S. Dollar Index", ylab="")
```




```r
plot(ind_prod_ts, main="Industrial Production Index", ylab="")
```



```r
plot(opec_prod_ts, main="OPEC Oil Production Index", ylab="")
```

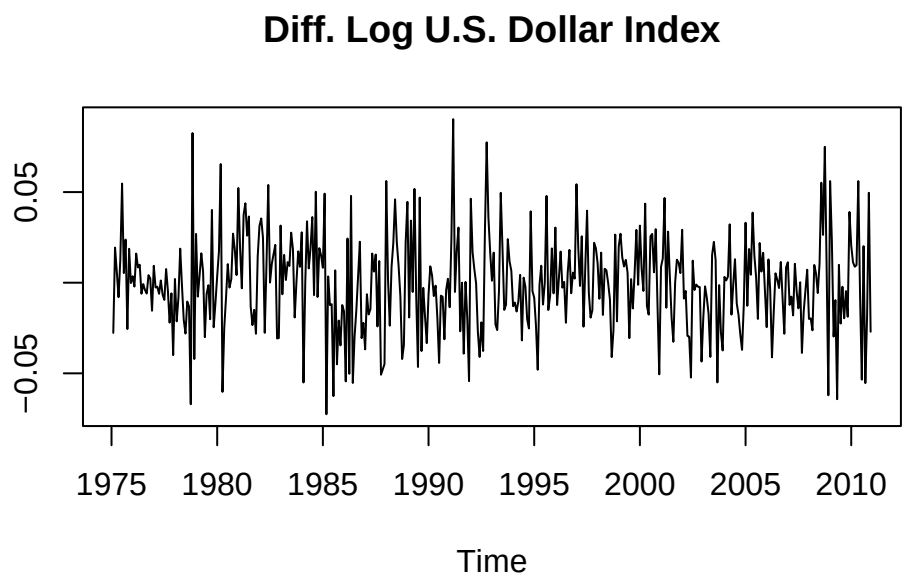


Transform log-differencing

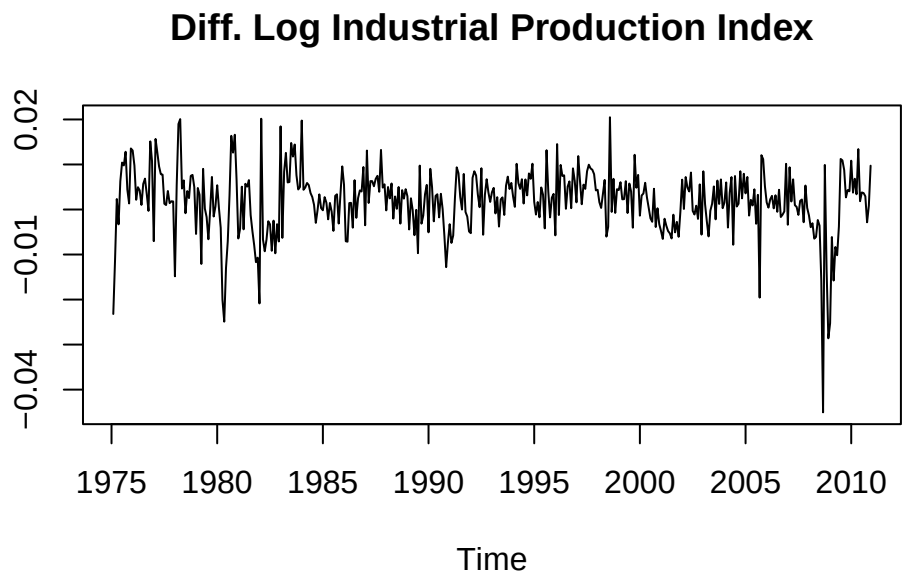
```r
d_usd_ts = diff(usd_idx_ts)
dl_usd_idx_ts = diff(log(usd_idx_ts))
dl_usd_idx_train_ts = diff(log(usd_idx_train_ts))

d_ind_ts = diff(ind_prod_ts)
dl_ind_ts = diff(log(ind_prod_ts))
dl_ind_train_ts = diff(log(ind_prod_train_ts))

d_opec_ts = diff(opec_prod_ts)
dl_opec_prod_ts = diff(log(opec_prod_ts))
dl_opec_idx_train_ts = diff(log(opec_prod_train_ts))
```

Plot

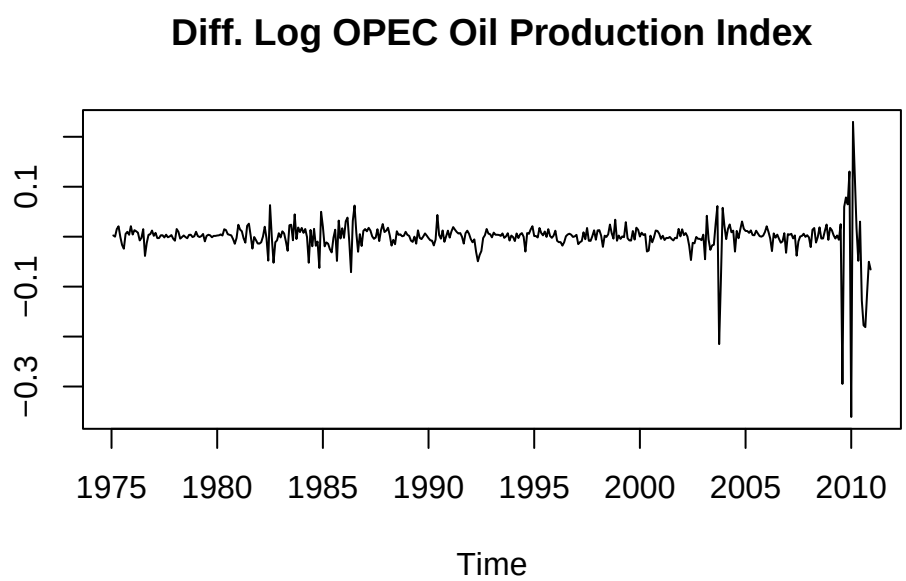
```r
#plot(dl_usd_idx_train_ts, main="Difference U.S. Dollar Index", ylab="")
plot(dl_usd_idx_train_ts, main="Diff. Log U.S. Dollar Index", ylab="")
```




```r
#plot(d_ind_train_ts, main="Difference Industrial Production Index", ylab="")
plot(dl_ind_train_ts, main="Diff. Log Industrial Production Index", ylab="")
```

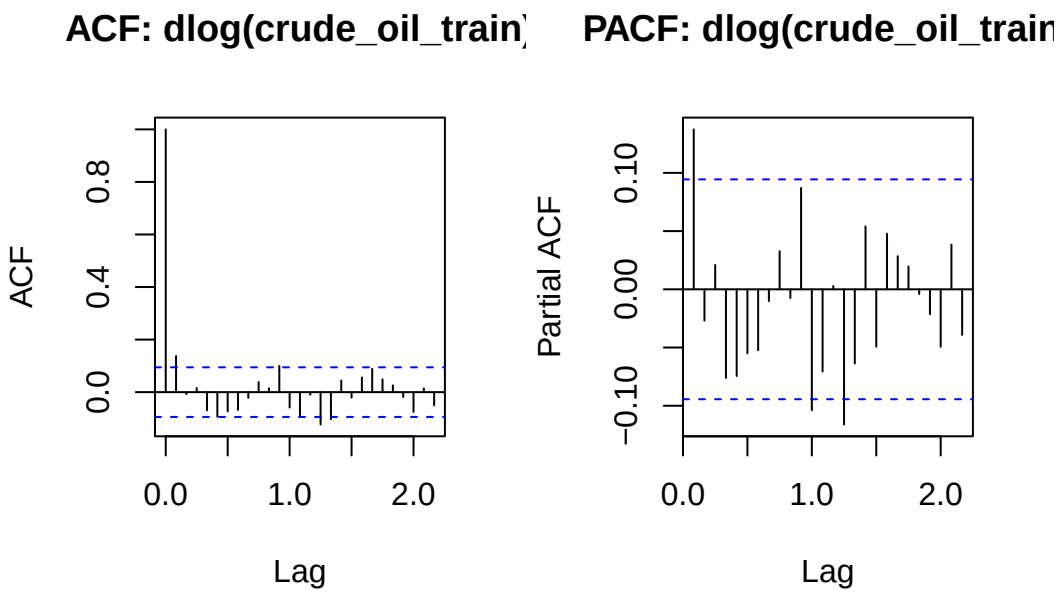



```
#plot(d_opec_idx_train_ts, main="Difference Industrial Production Index", ylab="")
plot(dl_opec_idx_train_ts, main="Diff. Log OPEC Oil Production Index", ylab="")
```



Diagnostics: variogram, ACF, PACF

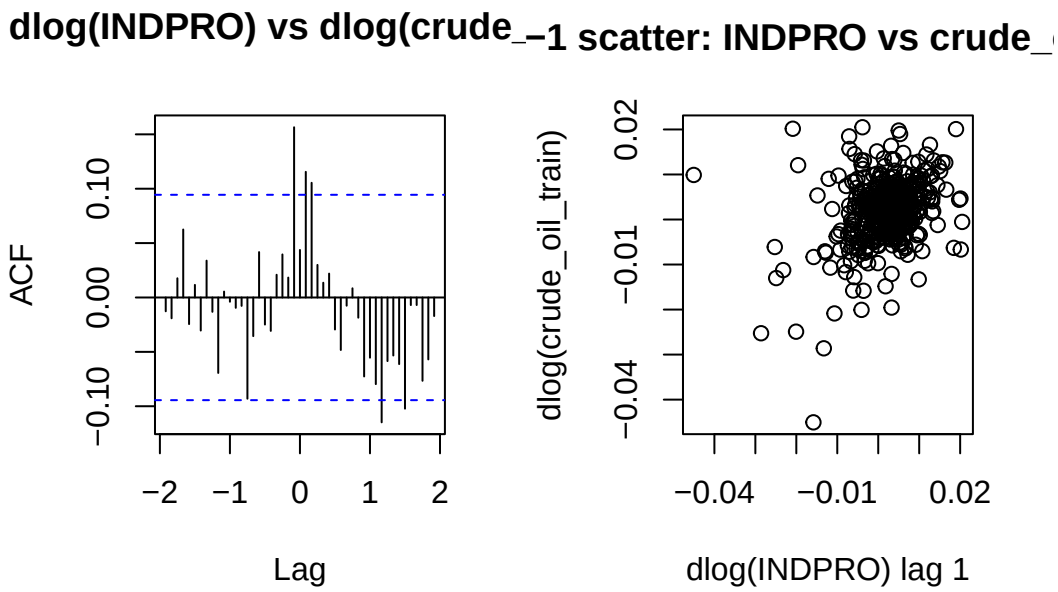
```
par(mfrow=c(1,2))
acf(dl_crude_oil_train_ts,
    main = "ACF: dlog(crude_oil_train)")
pacf(dl_crude_oil_train_ts,
     main = "PACF: dlog(crude_oil_train)")
```



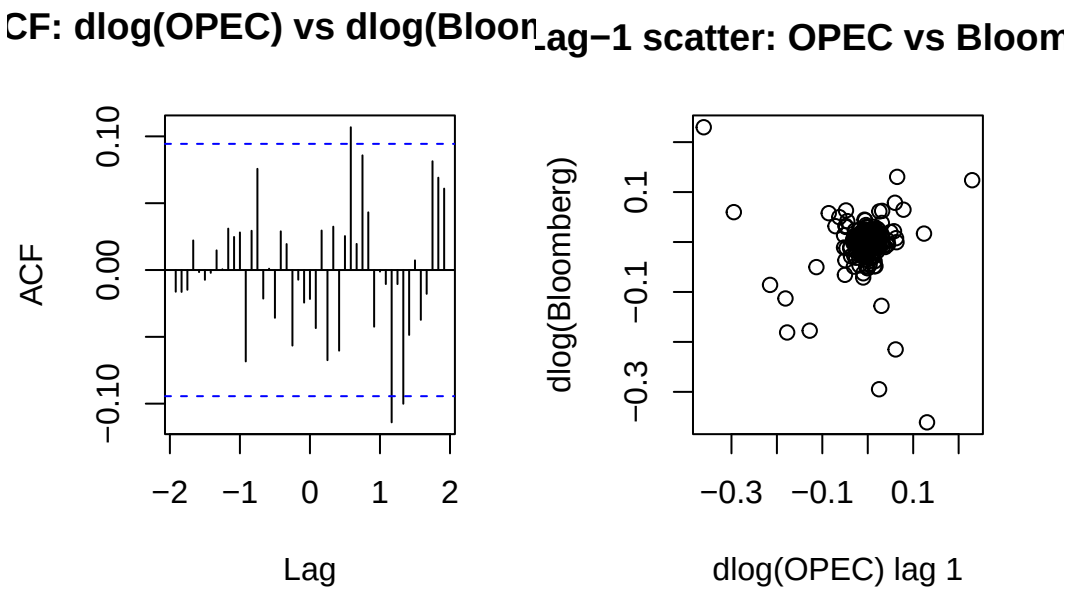
ARMAX

Cross-correlation diagnostics for leading variables

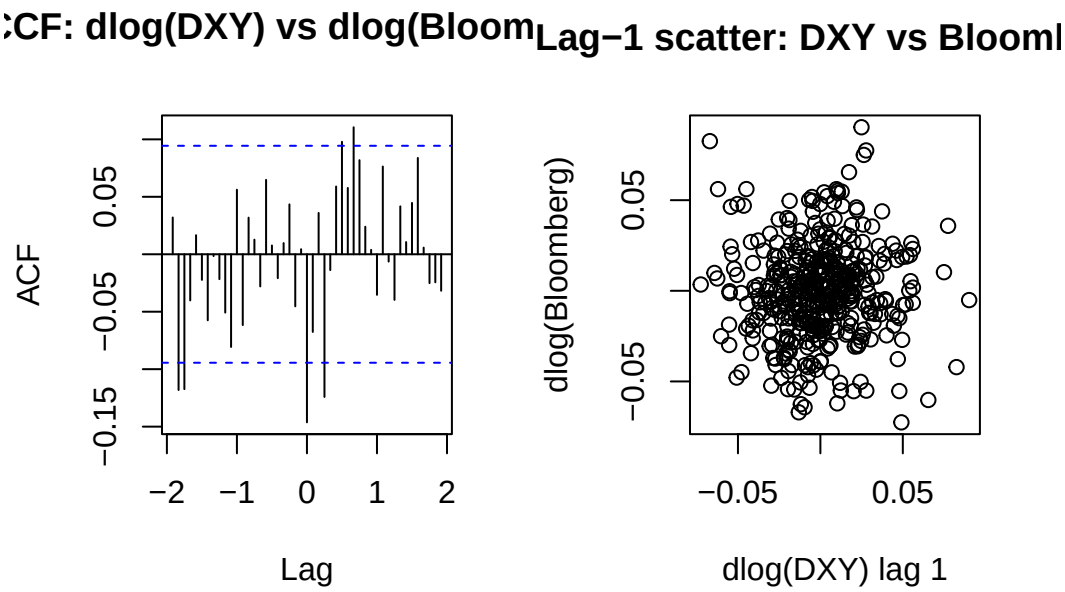
```
par(mfrow=c(1,2))
ccf(dl_ind_train_ts, dl_crude_oil_train_ts, main="CCF: dlog(INDPRO) vs dlog(crude_oil_train)"
plot(stats::lag(dl_ind_train_ts, -1), dl_ind_train_ts,
     xlab="dlog(INDPRO) lag 1", ylab="dlog(crude_oil_train)",
     main="Lag-1 scatter: INDPRO vs crude_oil_train")
```




```
par(mfrow=c(1,2))
ccf(dl_opec_idx_train_ts, dl_crude_oil_train_ts, main="CCF: dlog(OPEC) vs dlog(Bloomberg)")
plot(stats::lag(dl_opec_idx_train_ts, -1), dl_opec_idx_train_ts,
     xlab="dlog(OPEC) lag 1", ylab="dlog(Bloomberg)",
     main="Lag-1 scatter: OPEC vs Bloomberg")
```




```
par(mfrow=c(1,2))
ccf(dl_usd_idx_train_ts, dl_crude_oil_train_ts, main="CCF: dlog(DXY) vs dlog(Bloomberg)")
plot(stats::lag(dl_usd_idx_train_ts, -1), dl_usd_idx_train_ts,
     xlab="dlog(DXY) lag 1", ylab="dlog(Bloomberg)",
     main="Lag-1 scatter: DXY vs Bloomberg")
```



Align data

```
df_all = ts.intersect(
  oil  = dl_crude_oil_ts,
  ind  = dl_ind_ts,
  opec = dl_opec_prod_ts,
  usd  = dl_usd_idx_ts
)

start(df_all)
```

```
[1] 1971    1
```

```
end(df_all)
```

```
[1] 2026    2
```

Split Train/Holdout

```r
train_all = window(df_all, start = c(1975, 2), end = c(2010, 12))
holdout_all = window(df_all, start = c(2011, 1), end = c(2026, 1))

y_train = train_all[, "oil"]
y_holdout = holdout_all[, "oil"]

xreg_train = as.matrix(train_all[, c("ind", "opec", "usd")])
xreg_holdout = as.matrix(holdout_all[, c("ind", "opec", "usd")])
```

ARIMA baseline

```r
# ARIMA baseline
arima_model = auto.arima(
  y_train,
  seasonal = FALSE,
  stationary = TRUE
)

print(arima_model)
```

```
Series: y_train
ARIMA(0,0,1) with zero mean

Coefficients:
         ma1
      0.1459
s.e.  0.0486

sigma^2 = 0.007729:  log likelihood = 436.85
AIC=-869.7   AICc=-869.68   BIC=-861.57
```

```r
arima_res = arima_rolling_fc(arima_model, y_holdout)
```

ARMAX candidate model

```r
armax_model = auto.arima(
  y_train,
  xreg = xreg_train,
  seasonal = FALSE,
  stationary = TRUE
)

print(armax_model)
```

```
Series: y_train
Regression with ARIMA(0,0,1) errors

Coefficients:
         ma1     ind     opec      usd
      0.1301  0.3711  -0.0250  -0.4817
s.e.  0.0505  0.6032   0.1128   0.1609

sigma^2 = 0.007621:  log likelihood = 441.4
AIC=-872.8   AICc=-872.66   BIC=-852.47
```

extract nonseasonal ARMA order

```r
order_armax = c(armax_model$arma[1], 0, armax_model$arma[2])
include_mean_armax = any(names(armax_model$coef) %in% c("intercept", "mean"))
```

Rolling 1-step ARMAX

```r
armax_res = rolling_armax_fixed(
  y_train = y_train,
  x_train = xreg_train,
  y_holdout = y_holdout,
  x_holdout = xreg_holdout,
  order = order_armax,
  include_mean = include_mean_armax
)
```

RMSE comparison (ARIMA vs ARMAX)

```r
cat("ARIMA RMSE:", arima_res$rmse, "\n")
```

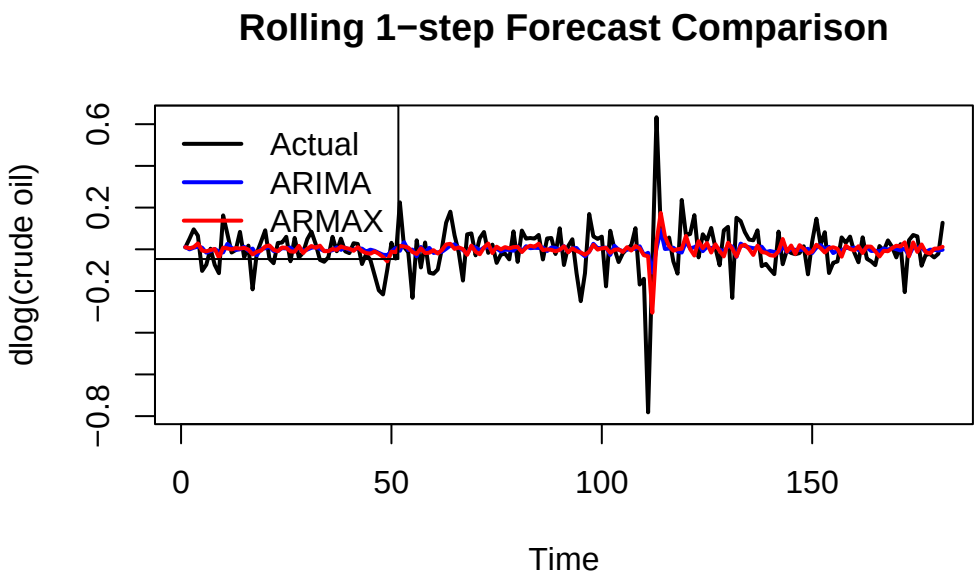
```
ARIMA RMSE: 0.1131622
```

```r
cat("ARMAX RMSE:", armax_res$rmse, "\n")
```

```
ARMAX RMSE: 0.1117057
```

Plot forecasts

```r
plot(as.numeric(y_holdout), type = "l", col = "black", lwd = 2,
     main = "Rolling 1-step Forecast Comparison",
     ylab = "dlog(crude oil)", xlab = "Time")

lines(as.numeric(arima_res$fc), col = "blue", lwd = 2)
lines(as.numeric(armax_res$fc), col = "red", lwd = 2)

legend("topleft",
       legend = c("Actual", "ARIMA", "ARMAX"),
       col = c("black", "blue", "red"),
       lty = 1, lwd = 2)
```



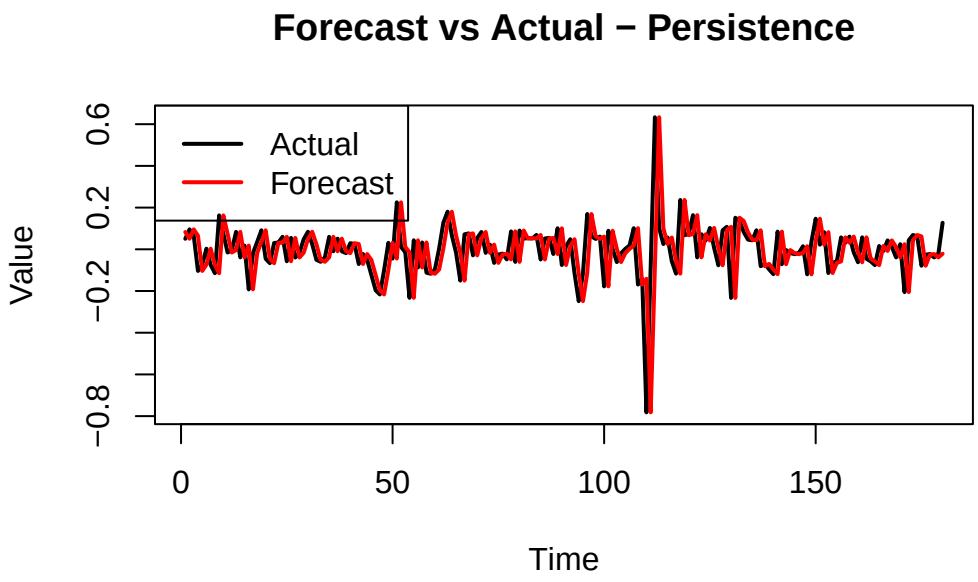
Simple Forecasting Rules

Persistence

```
persist_result = persist_fc(dl_crude_oil_train_ts, dl_crude_oil_holdout_ts)
persist_result$rmse
```

```
[1] 0.1506962
```

```
plot_forecast(dl_crude_oil_holdout_ts, persist_result$fc, "Persistence")
```

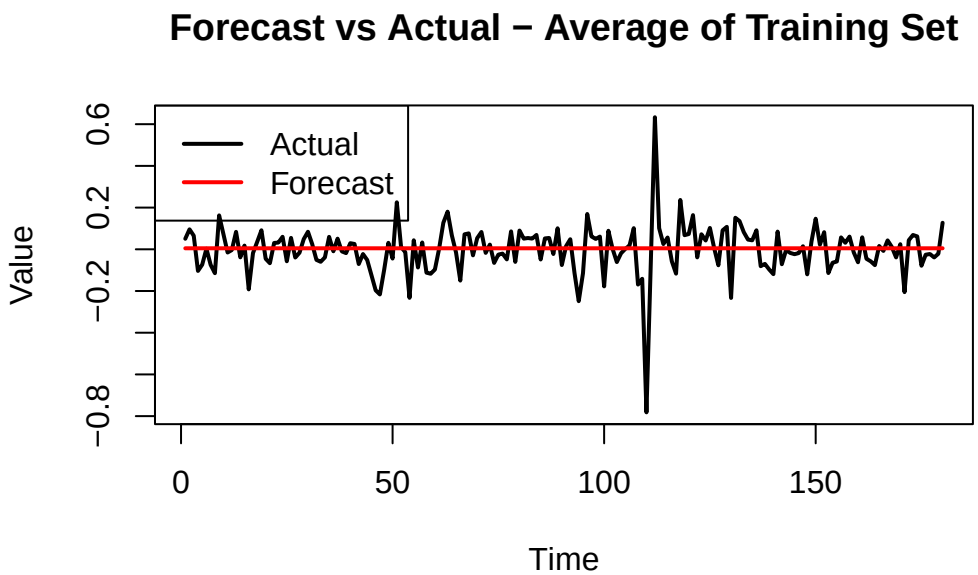


Average of Training Set

```
avg_result = avg_fc(dl_crude_oil_train_ts, dl_crude_oil_holdout_ts)
avg_result$rmse
```

```
[1] 0.115133
```

```
plot_forecast(dl_crude_oil_holdout_ts, avg_result$fc, "Average of Training Set")
```

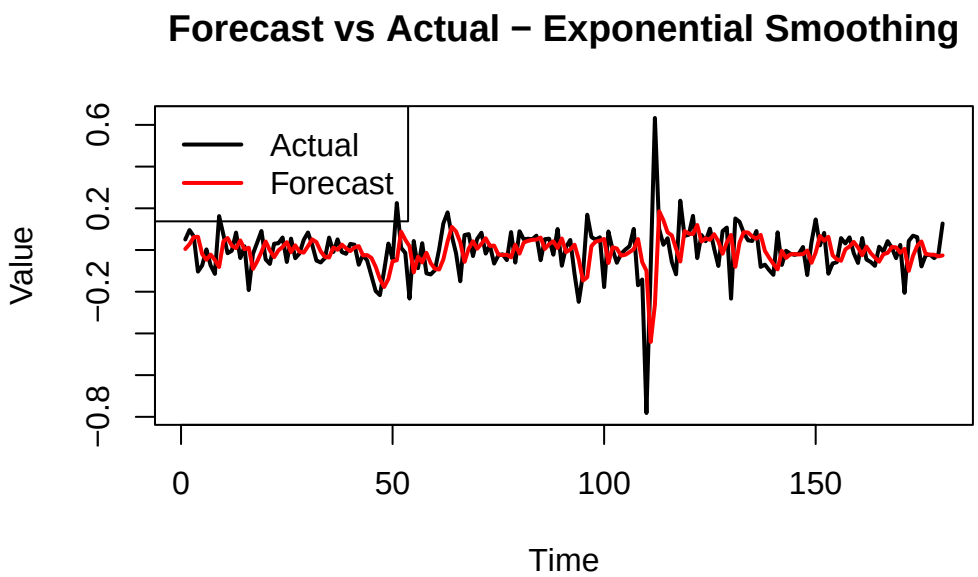


Exponential Smoothing

```
level = mean(dl_crude_oil_train_ts)
alpha = 0.5
esm_result = esm_fc(dl_crude_oil_train_ts, dl_crude_oil_holdout_ts, alpha, level)
esm_result$rmse
```

```
[1] 0.1323217
```

```r
plot_forecast(dl_crude_oil_holdout_ts, esm_result$fc, "Exponential Smoothing")
```



Linear in Previous Observation

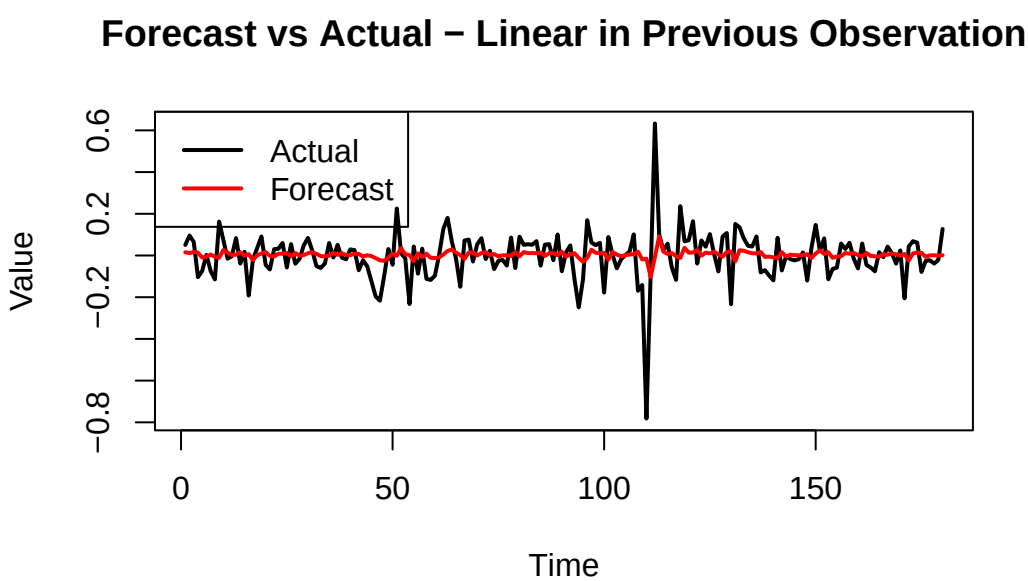
```r
n_train = length(dl_crude_oil_train_ts)
yt = dl_crude_oil_train_ts[2:n_train]
yt1 = dl_crude_oil_train_ts[1:(n_train - 1)]
lm_fit = lm(yt ~ yt1)
print(lm_fit)
```

```
Call:
lm(formula = yt ~ yt1)

Coefficients:
(Intercept)          yt1
   0.004227     0.137609
```

```r
linreg_result = linreg_fc(
  train_last = dl_crude_oil_train_ts[n_train],
  holdout = dl_crude_oil_holdout_ts,
  b0 = coef(lm_fit)[1],
  b1 = coef(lm_fit)[2]
)
linreg_result$rmse
```

```
[1] 0.1139725
```

```r
plot_forecast(dl_crude_oil_holdout_ts, linreg_result$fc,
              "Linear in Previous Observation")
```



Comparison

```r
# Rolling RMSE + AIC

model_names = c(
  "ARMA (diff)",
```

```
  "ARMA (log diff - auto)",
  "ARMA (log diff - manual)",
  "ARIMA (baseline)",
  "ARMAX",
  "Persistence",
  "Average",
  "Exponential Smoothing",
  "Linear Regression"
)

rmse_values = c(
  dwti_arma_rolling$rmse,
  dlwti_arma_rolling$rmse,
  dlwti_arma_manual_rolling$rmse,
  arima_res$rmse,
  armax_res$rmse,
  persist_result$rmse,
  avg_result$rmse,
  esm_result$rmse,
  linreg_result$rmse
)

aic_values = c(
  AIC(dwti_arma),
  AIC(dlwti_arma),
  AIC(dlwti_arma_manual),
  AIC(arima_model),
  AIC(armax_model),
  NA,
  NA,
  NA,
  NA
)

results_table = data.frame(
  Model = model_names,
  Rolling_RMSE = rmse_values,
  AIC = aic_values
)

print(results_table)
```

```
                     Model Rolling_RMSE       AIC
1              ARMA (diff)    6.5500040 2298.7546
2   ARMA (log diff - auto)    0.1134567 -869.7040
3 ARMA (log diff - manual)    0.1136074 -868.7228
4         ARIMA (baseline)    0.1131622 -869.7040
5                    ARMAX    0.1117057 -872.8009
6              Persistence    0.1506962        NA
7                  Average    0.1151330        NA
8    Exponential Smoothing    0.1323217        NA
9        Linear Regression    0.1139725        NA
```

Saving Data

```
# Saving data

write.table(usd_idx_train_ts, "savedData/usd_idx_train_ts_new.csv", sep = ",", row.names = FA
write.table(usd_idx_holdout_ts, "savedData/usd_idx_holdout_ts_new.csv", sep = ",", row.names

write.table(ind_prod_train_ts, "savedData/ind_prod_train_ts_new.csv", sep = ",", row.names =
write.table(ind_prod_holdout_ts, "savedData/ind_prod_holdout_ts_new.csv", sep = ",", row.name
write.table(crude_oil_ts, "savedData/crude_oil_ts_new.csv", sep = ",", row.names = FALSE)

write.table(opec_prod_train_ts, "savedData/opec_prod_train_ts_new.csv", sep = ",", row.names
write.table(opec_prod_holdout_ts, "savedData/opec_prod_holdout_ts_new.csv", sep = ",", row.na

write.table(d_usd_ts, "savedData/d_usd_ts_new.csv", sep = ",", row.names = FALSE)
write.table(dl_usd_idx_train_ts, "savedData/dl_usd_idx_train_ts_new.csv", sep = ",", row.name
write.table(d_ind_ts, "savedData/d_ind_ts_new.csv", sep = ",", row.names = FALSE)
write.table(dl_ind_train_ts, "savedData/dl_ind_train_ts_new.csv", sep = ",", row.names = FALS
write.table(d_opec_ts, "savedData/d_opec_ts_new.csv", sep = ",", row.names = FALSE)
write.table(dl_opec_idx_train_ts, "savedData/dl_opec_idx_train_ts_new.csv", sep = ",", row.na
```